



Building hexagonal architectures on AWS

AWS Prescriptive Guidance



AWS Prescriptive Guidance: Building hexagonal architectures on AWS

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

Introduction	1
Overview	3
Domain-driven design (DDD)	3
Hexagonal architecture	3
Targeted business outcomes	5
Improving the development cycle	6
Testing in the cloud	6
Testing locally	6
Parallelization of development	7
Product time to market	7
Quality by design	8
Localized changes and increased readability	8
Testing business logic first	8
Maintainability	9
Adapting to change	10
Adapting to new non-functional requirements by using ports and adapters	10
Adapting to new business requirements by using commands and command handlers	10
Decoupling components by using the service façade or CQRS pattern	11
Organizational scaling	12
Best practices	14
Model the business domain	14
Write and run tests from the beginning	14
Define the behavior of the domain	15
Automate testing and deployment	15
Scale your product by using microservices and CQRS	15
Design a project structure that maps to hexagonal architecture concepts	16
Infrastructure examples	18
Start simple	18
Apply the CQRS pattern	19
Evolve the architecture by adding containers, a relational database, and an external API	20
Add more domains (zoom out)	21
FAQ	23
Why should I use a hexagonal architecture?	23
Why should I use domain-driven design?	23

Can I practice test-driven development without hexagonal architecture?	23
Can I scale my product without hexagonal architecture and domain-driven design?	23
Which technologies should I use to implement hexagonal architecture?	23
I am developing a minimum viable product. Does it make sense to spend time thinking about software architecture?	24
I am developing a minimum viable product and have no time to write tests.	24
Which additional design patterns can I use with hexagonal architecture?	24
Next steps	25
Resources	26
Document history	28

Building hexagonal architectures on AWS

Furkan Oruc, Dominik Goby, Darius Kuncze, and Michal Ploski, Amazon Web Services (AWS)

June 2022 ([document history](#))

This guide describes a mental model and a collection of patterns for developing software architectures. These architectures are easy to maintain, extend, and scale across the organization as product adoption grows. Cloud hyperscalers such as Amazon Web Services (AWS) provide building blocks for small and large enterprises to innovate and create new software products. The rapid pace of these new service and feature introductions leads business stakeholders to expect their development teams to prototype new minimum viable products (MVPs) faster, so that new ideas can be tested and verified as soon as possible. Often, those MVPs are adopted and become part of the enterprise software ecosystem. In the process of producing these MVPs, teams sometimes abandon software development rules and best practices, such as [SOLID principles](#) and unit testing. They assume that this approach will speed up development and reduce the time to market. However, if they fail to create a foundational model and a framework for software architecture at all levels, it will be difficult or even impossible to develop new features for the product. Lack of certainty and changing requirements can also slow down the team during the development process.

This guide walks through a proposed software architecture, from a low-level hexagonal architecture to a high-level architectural and organizational decomposition, that uses domain-driven design (DDD) to address these challenges. DDD helps manage business complexity and scale the engineering team as new features are developed. It aligns business and technical stakeholders to the business problems, called *domains*, by using ubiquitous language. Hexagonal architecture is a technical enabler of this approach in a very specific domain, called a *bounded context*. A bounded context is a highly cohesive and loosely coupled sub-area of the business problem. We recommend that you adopt hexagonal architecture for all your enterprise software projects regardless of their complexity.

Hexagonal architecture encourages the engineering team to solve the business problem first, whereas classical layered architecture shifts engineering focus away from the domain to solving technical problems first. Furthermore, if software follows a hexagonal architecture, it's easier to adopt a [test-driven development approach](#), which reduces the feedback loop that developers need to test business requirements. Lastly, using [commands and command handlers](#) is a way to apply the single responsibility and open-closed principles from SOLID. Adhering to these principles

produces a code base that developers and architects working on the project can easily navigate and understand, and reduces the risk of introducing breaking changes to existing functionality.

This guide is for software architects and developers who are interested in understanding the benefits of adopting hexagonal architecture and DDD for their software development projects. It includes an example of designing an infrastructure for your application on AWS that supports hexagonal architecture. For an example implementation, see [Structure a Python project in hexagonal architecture using AWS Lambda](#) on the AWS Prescriptive Guidance website.

Overview

Domain-driven design (DDD)

In [domain-driven design \(DDD\)](#), a domain is the core of the software system. The domain model is defined first, before you develop any other module, and it doesn't depend on other low-level modules. Instead, modules such as databases, the presentation layer, and external APIs all depend on the domain.

In DDD, architects decompose the solution into bounded contexts by using business logic-based decomposition instead of technical decomposition. The benefits of this approach are discussed in the [Targeted business outcomes](#) section.

DDD is easier to implement when teams use hexagonal architecture. In hexagonal architecture, the application core is the center of the application. It is decoupled from other modules through ports and adapters, and has no dependencies on other modules. This aligns perfectly with DDD, where a domain is the core of the application that solves a business problem. This guide proposes an approach where you model the core of the hexagonal architecture as the domain model of a bounded context. The next section describes hexagonal architecture in more detail.

This guide doesn't cover all aspects of DDD, which is a very broad topic. To gain a better understanding, you can review the DDD resources listed on the [Domain Language](#) website.

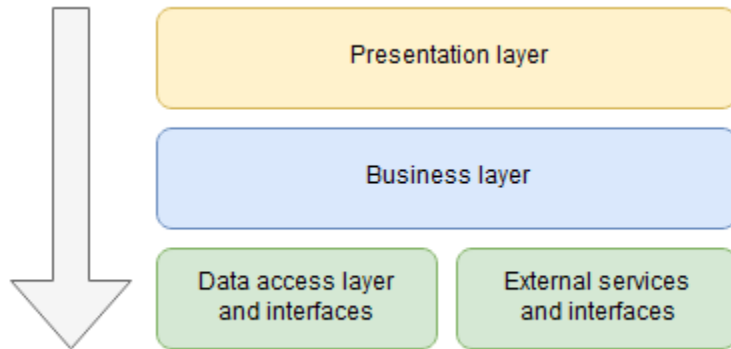
Hexagonal architecture

Hexagonal architecture, also known as *ports and adapters* or *onion architecture*, is a principle of managing dependency inversion in software projects. Hexagonal architecture promotes a strong focus on the core domain business logic when developing software, and treats external integration points as secondary. Hexagonal architecture helps software engineers adopt good practices such as test-driven development (TDD), which, in turn, promotes [architecture evolution](#), and helps you manage complex domains in the long term.

Let's compare hexagonal architecture to classical layered architecture, which is the most popular choice for modeling structured software projects. There are subtle but powerful differences between the two approaches.

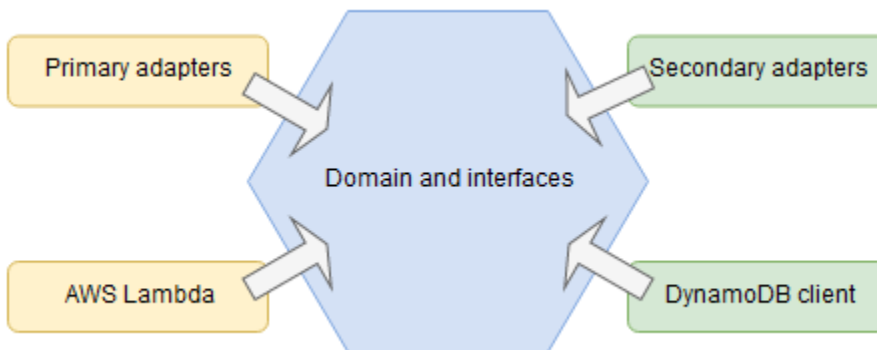
In layered architecture, software projects are structured in tiers, which represent broad concerns such as business logic or presentation logic. This architecture uses a dependency hierarchy, where

the top layers have dependencies on the layers below them, but not the other way around. In the following diagram, the presentation layer is responsible for user interactions, so it includes the user interface, APIs, command line interfaces, and similar components. The presentation layer has a dependency on the business layer, which implements domain logic. The business layer, in turn, has dependencies on the data access layer and on multiple external services.



The main disadvantage of this configuration is the dependency structure. For example, if the model for storing data in the database changes, this affects the data access interface. Any change to the data model also affects the business layer, which has a dependency on the data access interface. As a result, software engineers cannot make any infrastructure changes without affecting the domain logic. This, in turn, increases the likelihood of regression bugs.

Hexagonal architecture defines dependency relationships in a different way, as illustrated in the following diagram. It concentrates decision making around domain business logic, which defines all the interfaces. External components interact with the business logic through interfaces called *ports*. Ports are abstractions that define the interactions of the domain with the external world. Each infrastructure component must implement those ports, so changes in those components no longer affect the core domain logic.



The surrounding components are called *adapters*. An adapter is a proxy between the external world and the internal world, and implements a port defined in the domain. Adapters can be categorized in two groups: primary and secondary. Primary adapters are the entry points to the software component. They allow external actors, users, and services to interact with the core logic. AWS Lambda is a good example of a primary adapter. It integrates with multiple AWS services that can invoke the Lambda functions as entry points. Secondary adapters are external service library wrappers that handle communications with the external world. A good example of a secondary adapter is an Amazon DynamoDB client for data access.

Targeted business outcomes

The hexagonal architecture discussed in this guide helps you achieve the following objectives:

- [Reduce time to market by improving the development cycle](#)
- [Improve software quality](#)
- [Adapt more easily to change](#)

These processes are discussed in detail in the following sections.

Improving the development cycle

Developing software for the cloud introduces new challenges for software engineers, because it is very hard to replicate the runtime environment locally on the development machine. A straightforward way to validate the software is to deploy it to the cloud and test it there. However, this approach involves a lengthy feedback cycle, especially when the software architecture contains multiple serverless deployments. Improving this feedback cycle shortens the time to develop features, which reduces the time to market significantly.

Testing in the cloud

Testing directly in the cloud is the only way to make sure that your architectural components, such as gateways in Amazon API Gateway, AWS Lambda functions, Amazon DynamoDB tables, and AWS Identity and Access Management (IAM) permissions, are configured correctly. It might also be the only reliable way to test component integrations. Although some AWS services (such as [DynamoDB](#)) can be deployed locally, most of them cannot be replicated in a local setup. At the same time, third-party tools such as [Moto](#) and [LocalStack](#) that mock AWS services for testing purposes might not reflect real service API contracts accurately, or the number of features might be limited.

However, the most complex part of the enterprise software is in the business logic, not in the cloud architecture. The architecture changes less often than the domain, which must accommodate new business requirements. Thus, testing business logic in the cloud becomes an intense process of making a change in the code, initiating a deployment, waiting for the environment to be ready, and validating the change. If a deployment takes as little as 5 minutes, making and testing 10 changes in the business logic will take an hour or more. If business logic is more complex, testing might require days of just waiting for deployments to complete. If you have multiple features and engineers on the team, the extended period quickly becomes noticeable to the business.

Testing locally

A hexagonal architecture helps developers focus on the domain instead of infrastructure technicalities. This approach uses local tests (the unit testing tools in your chosen development framework) to cover domain logic requirements. You won't have to spend time solving technical integration problems or deploy your software to the cloud to test the business logic. You can run unit tests locally and reduce the feedback loop from minutes to seconds. If a deployment takes 5

minutes but unit tests complete in 5 seconds, that's a significant reduction in the time it takes to detect mistakes. The [Testing business logic first](#) section later in this guide covers this approach in more detail.

Parallelization of development

The hexagonal architecture approach enables development teams to parallelize development efforts. Developers can design and implement different components of the service individually. This parallelization is possible through the isolation of each component and the defined interfaces between each component.

Product time to market

Local unit testing improves the development feedback cycle and reduces the time to market for new products or features, especially when these contain complex business logic, as explained previously. Furthermore, increased code coverage by unit tests significantly reduces the risk of introducing regression bugs when you update or refactor the code base. Unit test coverage also enables you to continuously refactor the code base to keep it well organized, which speeds the onboarding process for new engineers. This is discussed further in the [Quality by design](#) section. Lastly, if the business logic is well isolated and tested, it enables developers to adapt quickly to changing functional and non-functional requirements. This is explained further in the [Adapting to change](#) section.

Quality by design

Adopting a hexagonal architecture helps promote the quality of your code base from the start of your project. It is important to build a process that helps you meet expected quality requirements from the beginning, without slowing down the development process.

Localized changes and increased readability

Using the hexagonal architecture approach enables developers to change code in one class or component without affecting other classes or components. This design promotes the cohesion of developed components. By decoupling the domain from adapters and using well-known interfaces, you can increase the readability of the code. It becomes easier to identify issues and corner cases.

This approach also facilitates code review during development and limits the introduction of undetected changes or technical debt.

Testing business logic first

Local testing can be accomplished by introducing end-to-end, integration, and unit tests to the project. End-to-end tests cover the whole incoming request lifecycle. They typically invoke an application entry point and test to see if it has accomplished the business requirement. Each software project should have at least one test scenario that uses known inputs and produces expected outputs. However, adding more corner-case scenarios can get complex, because each test must be configured to send a request through an entry point (for example, through a REST API or queues), go through all the integration points that the business action requires, and then assert the result. Setting up the environment for the test scenario and asserting results can take a lot of developers' time.

In hexagonal architecture, you test business logic in isolation, and use integration tests to test secondary adapters. You can use mock or fake adapters in your business logic tests. You can also combine the tests for business use cases with unit tests for your domain model to maintain high coverage with low coupling. As a good practice, integration tests should not validate business logic. Instead, they should verify that the secondary adapter calls the external services correctly.

Ideally, you can use test-driven development (TDD) and start defining domain entities or business use cases with proper tests at the very beginning of development. Writing the tests first helps you create mock implementations of the interfaces required by the domain. When the tests

are successful and domain logic rules are satisfied, you can implement the actual adapters and deploy software to the test environment. At this point, your implementation of domain logic might not be ideal. You can then work on refactoring the existing architecture to evolve it by introducing design patterns or rearranging code in general. By using this approach, you can avoid introducing regression bugs and you can improve the architecture as the project grows. By combining this approach with the automatic tests you run in your continuous integration process, you can decrease the number of potential bugs before they get to production.

If you use serverless deployments, you can quickly provision an instance of the application in your AWS account for manual integration and end-to-end testing. After these implementation steps, we recommend that you automate testing with every new change pushed to the repository.

Maintainability

Maintainability refers to operating and monitoring an application to ensure that it meets all requirements and minimizing the probability of a system failure. To make the system operable, you must adapt it to future traffic or operational requirements. You must also ensure that it's available and easy to deploy with minimum or no impact on clients.

To understand the current and historical state of your system, you must make it observable. You can do this by providing specific metrics, logs, and traces that operators can use to ensure that the system works as expected and to track bugs. These mechanisms should also allow operators to conduct root cause analysis without having to log in to the machine and read the code.

A hexagonal architecture aims to increase the maintainability of your web applications so that your code requires less work overall. By separating modules, localizing changes, and decoupling application business logic from adapter implementation, you can produce metrics and logs that help operators gain a deep understanding of the system and understand the scope of specific changes made to the primary or secondary adapters.

Adapting to change

Software systems tend to get complicated. One reason for this could be frequent changes to the business requirements and little time to adapt the software architecture accordingly. Another reason could be insufficient investment for setting up the software architecture at the beginning of the project to adapt to frequent changes. Whatever the reason, a software system could get complicated to the point where it is almost impossible to make a change. Therefore, it is important to build maintainable software architecture from the beginning of the project. Good software architecture can adapt to changes easily.

This section explains how to design maintainable applications by using hexagonal architecture that adapts easily to non-functional or business requirements.

Adapting to new non-functional requirements by using ports and adapters

As the core of the application, the domain model defines the actions that are required from the outside world to fulfill business requirements. These actions are defined through abstractions, which are called *ports*. These ports are implemented by separate adapters. Each adapter is responsible for an interaction with another system. For example, you might have one adapter for the database repository and another adapter for interacting with a third-party API. The domain is not aware of the adapter implementation, so it is easy to replace one adapter with another. For example, the application might switch from a SQL database to a NoSQL database. In this case, a new adapter has to be developed to implement the ports that are defined by the domain model. The domain has no dependencies on the database repository and uses abstractions to interact, so there would be no need to change anything in the domain model. Therefore, hexagonal architecture adapts to non-functional requirements with ease.

Adapting to new business requirements by using commands and command handlers

In classical layered architecture, the domain depends on the persistence layer. If you want to change the domain, you would also have to change the persistence layer. In comparison, in hexagonal architecture, the domain doesn't depend on other modules in the software. The domain is the core of the application, and all other modules (ports and adapters) depend on the domain

model. The domain uses the [dependency inversion principle](#) to communicate with the outside world through ports. The benefit of dependency inversion is that you can change the domain model freely without being afraid to break other parts of the code. Because the domain model reflects the business problem that you are trying to solve, updating the domain model to adapt to changing business requirements isn't a problem.

When you develop software, separation of concerns is an important principle to follow. To achieve this separation, you can use a [slightly modified command pattern](#). This is a behavioral design pattern in which all required information to complete an operation is encapsulated in a command object. These operations are then processed by command handlers. Command handlers are methods that receive a command, alter the state of the domain, and then return a response to the caller. You can use different clients, such as synchronous APIs or asynchronous queues, to run commands. We recommend that you use commands and command handlers for every operation on the domain. By following this approach, you can add new features by introducing new commands and command handlers, without changing your existing business logic. Thus, using a command pattern makes it easier to adapt to new business requirements.

Decoupling components by using the service façade or CQRS pattern

In hexagonal architecture, primary adapters are responsible for loosely coupling incoming read and write requests from clients to the domain. There are two ways to achieve this loose coupling: by using a service façade pattern or by using the command query responsibility segregation (CQRS) pattern.

The [service façade pattern](#) provides a front-facing interface to serve clients such as the presentation layer or a microservice. A service façade provides clients with several read and write operations. It's responsible for transferring incoming requests to the domain and mapping the response received from the domain to clients. Using a service façade is easy for microservices that have a single responsibility with several operations. However, when using the service façade, it is harder to follow [single responsibility and open-closed principles](#). The single responsibility principle states that each module should have responsibility over only a single functionality of the software. The open-closed principle states that code should be open for extension and closed for modification. As the service façade extends, all operations are collected in one interface, more dependencies are encapsulated into it, and more developers start modifying the same façade. Therefore, we recommend using a service façade only if it's clear that the service would not extend a lot during development.

Another way to implement primary adapters in hexagonal architecture is to use the [CQRS pattern](#), which separates read and write operations using queries and commands. As explained previously, commands are objects that contain all the information required to change the state of the domain. Commands are performed by command handler methods. Queries, on the other hand, do not alter the state of the system. Their only purpose is to return data to clients. In the CQRS pattern, commands and queries are implemented in separate modules. This is especially advantageous for projects that follow an [event-driven architecture](#), because a command could be implemented as an event that is processed asynchronously, whereas a query can be run synchronously by using an API. A query can also use a different database that is optimized for it. The disadvantage of the CQRS pattern is that it takes more time to implement than a service façade. We recommend using the CQRS pattern for projects that you plan to scale and maintain in the long term. Commands and queries provide an effective mechanism for applying the single responsibility principle and developing loosely coupled software, especially in large-scale projects.

CQRS has great benefits in the long term, but requires an initial investment. For this reason, we recommend that you evaluate your project carefully before you decide to use the CQRS pattern. However, you can structure your application by using commands and command handlers right from the start without separating read/write operations. This will help you easily refactor your project for CQRS if you decide to adopt that approach later.

Organizational scaling

A combination of hexagonal architecture, domain-driven design, and (optionally) CQRS enables your organization to quickly scale your product. According to [Conway's Law](#), software architectures tend to evolve to reflect a company's communication structures. This observation has historically had negative connotations, because big organizations often structure their teams based on technical expertise such as database, enterprise service bus, and so on. The problem with this approach is that product and feature development always involve crosscutting concerns, such as security and scalability, which require constant communication among teams. Structuring teams based on technical features creates unnecessary silos in the organization, which result in poor communications, lack of ownership, and losing sight of the big picture. Eventually, these organizational problems are reflected in the software architecture.

The [Inverse Conway Maneuver](#), on the other hand, defines the organizational structure based on domains that promote the software architecture. For example, cross-functional teams are given responsibility for a [specific set of bounded contexts](#), which are identified by using DDD and [event storming](#). Those bounded contexts might reflect very specific features of the product. For example,

the account team might be responsible for the payment context. Each new feature is assigned to a new team that has highly cohesive and loosely coupled responsibilities, so they can focus only on the delivery of that feature and reduce the time to market. Teams can be scaled according to the complexity of features, so complex features can be assigned to more engineers.

Best practices

Model the business domain

Work back from the business domain to the software design to ensure that the software you're writing fits the business need.

Use domain-driven design (DDD) methodologies such as [event storming](#) to model the business domain. Event storming has a flexible workshop format. During the workshop, domain and software experts explore the complexity of the business domain collaboratively. Software experts use the deliverables of the workshop to start the design and development process for software components.

Write and run tests from the beginning

Use test-driven development (TDD) to verify the correctness of the software that you are developing. TDD works best at a unit test level. The developer designs a software component by writing a test first, which invokes that component. That component has no implementation at the beginning, therefore the test fails. As a next step, the developer implements the component's functionality, using test fixtures with mock objects to simulate the behavior of external dependencies, or ports. When the test succeeds, the developer can continue by implementing real adapters. This approach improves software quality and results in more readable code, because developers understand how users would use the components. Hexagonal architecture supports the TDD methodology by separating the application core. Developers write unit tests that focus on the domain core behavior. They don't have to write complex adapters to run their tests; instead, they can use simple mock objects and fixtures.

Use behavior-driven development (BDD) to ensure end-to-end acceptance at a feature level. In BDD, developers define scenarios for features and verify them with business stakeholders. BDD tests use as much natural language as possible to achieve this. Hexagonal architecture supports the BDD methodology with its concept of primary and secondary adapters. Developers can create primary and secondary adapters that can run locally without calling external services. They configure the BDD test suite to use the local primary adapter to run the application.

Automatically run each test in the continuous integration pipeline to constantly evaluate the quality of the system.

Define the behavior of the domain

Decompose the domain into entities, value objects, and aggregates (read about [implementing domain-driven design](#)), and define their behavior. Implement the behavior of the domain so that tests that were written at the beginning of the project succeed. Define commands that invoke the behavior of domain objects. Define events that the domain objects emit after they complete a behavior.

Define interfaces that adapters can use to interact with the domain.

Automate testing and deployment

After an initial proof of concept, we recommend that you invest time implementing DevOps practices. For example, continuous integration and continuous delivery (CI/CD) pipelines and dynamic test environments help you maintain the quality of the code and avoid errors during deployment.

- Run your unit tests inside your CI process and test your code before it is merged.
- Build a CD process to deploy your application into a static dev/test environment or into dynamically created environments that support automatic integration and end-to-end testing.
- Automate the deployment process for dedicated environments.

Scale your product by using microservices and CQRS

If your product is successful, scale your product by decomposing your software project into microservices. Utilize the portability that hexagonal architecture provides to improve performance. Split query services and command handlers into separate synchronous and asynchronous APIs. Consider adopting the command query responsibility segregation (CQRS) pattern and event-driven architecture.

If you get many new feature requests, consider scaling your organization based on DDD patterns. Structure your teams in such a way that they own one or more features as bounded contexts, as discussed previously in the [Organizational scaling](#) section. These teams can then implement business logic by using hexagonal architecture.

Design a project structure that maps to hexagonal architecture concepts

Infrastructure as code (IaC) is a widely adopted practice in cloud development. It lets you define and maintain your infrastructure resources (such as networks, load balancers, virtual machines, and gateways) as source code. This way, you can track all changes to your architecture by using a version control system. In addition, you can create and move the infrastructure easily for testing purposes. We recommend that you keep your application code and infrastructure code in the same repository when you develop your cloud applications. This approach makes it easy to maintain infrastructure for your application.

We recommend that you divide your application into three folders or projects that map to the concepts of hexagonal architecture: `entrypoints` (primary adapters), `domain` (domain and interfaces), and `adapters` (secondary adapters).

The following project structure provides an example of this approach when designing an API on AWS. The project maintains application code (`app`) and infrastructure code (`infra`) in the same repository, as recommended earlier.

```
app/ # application code
|--- adapters/ # implementation of the ports defined in the domain
|   |--- tests/ # adapter unit tests
|--- entrypoints/ # primary adapters, entry points
|   |--- api/ # api entry point
|       |--- model/ # api model
|       |--- tests/ # end to end api tests
|--- domain/ # domain to implement business logic using hexagonal architecture
|   |--- command_handlers/ # handlers used to run commands on the domain
|   |--- commands/ # commands on the domain
|   |--- events/ # events emitted by the domain
|   |--- exceptions/ # exceptions defined on the domain
|   |--- model/ # domain model
|   |--- ports/ # abstractions used for external communication
|   |--- tests/ # domain tests
infra/ # infrastructure code
```

As discussed earlier, the domain is the core of the application and doesn't depend on any other module. We recommend that you structure the domain folder to include the following subfolders:

- `command_handlers` contains the methods or classes that run commands on the domain.

- `commands` contains the command objects that define the information required to perform an operation on the domain.
- `events` contains the events that are emitted through the domain and then routed to other microservices.
- `exceptions` contains the known errors defined within the domain.
- `model` contains the domain entities, value objects, and domain services.
- `ports` contains the abstractions through which the domain communicates with databases, APIs, or other external components.
- `tests` contains the test methods (such as business logic tests) that are run on the domain.

The primary adapters are the entry points to the application, as represented by the `entrypoints` folder. This example uses the `api` folder as the primary adapter. This folder contains an API `model`, which defines the interface the primary adapter requires to communicate with clients. The `tests` folder contains end-to-end tests for the API. These are shallow tests that validate that the components of the application are integrated and work in harmony.

The secondary adapters, as represented by the `adapters` folder, implement the external integrations required by the domain ports. A database repository is a great example of a secondary adapter. When the database system changes, you can write a new adapter by using the implementation that's defined by the domain. There is no need to change the domain or business logic. The `tests` subfolder contains external integration tests for each adapter.

Infrastructure examples on AWS

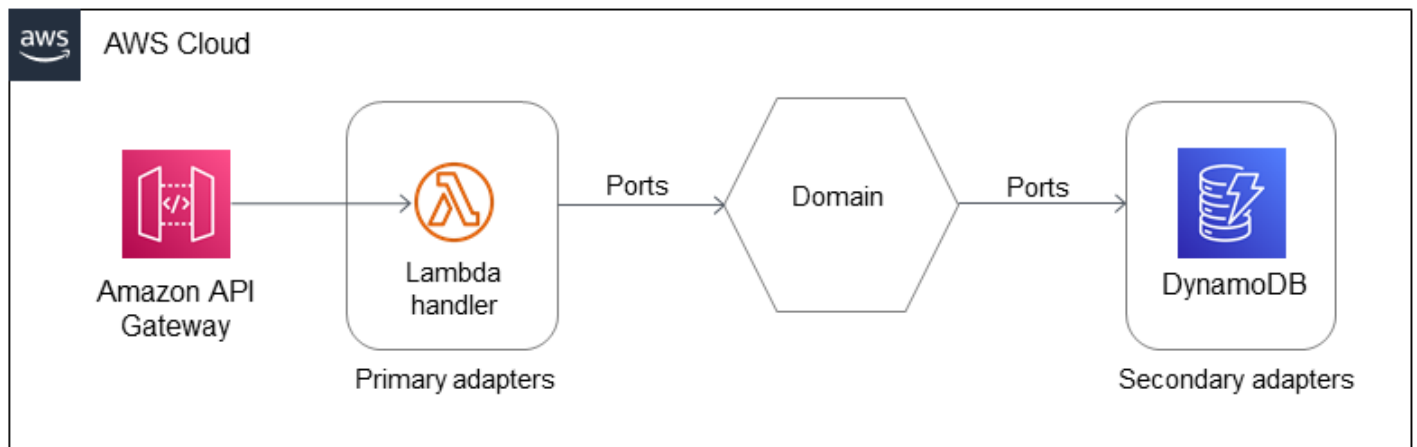
This section provides examples for designing an infrastructure for your application on AWS that you can use to implement a hexagonal architecture. We recommend that you start with a simple architecture for building a minimum viable product (MVP). Most microservices need a single entry point to handle client requests, a compute layer to run the code, and a persistence layer to store data. The following AWS services are great candidates for use as clients, primary adapters, and secondary adapters in hexagonal architecture:

- **Clients:** Amazon API Gateway, Amazon Simple Queue Service (Amazon SQS), Elastic Load Balancing, Amazon EventBridge
- **Primary adapters:** AWS Lambda, Amazon Elastic Container Service (Amazon ECS), Amazon Elastic Kubernetes Service (Amazon EKS), Amazon Elastic Compute Cloud (Amazon EC2)
- **Secondary adapters:** Amazon DynamoDB, Amazon Relational Database Service (Amazon RDS), Amazon Aurora, API Gateway, Amazon SQS, Elastic Load Balancing, EventBridge, Amazon Simple Notification Service (Amazon SNS)

The following sections discuss these services in the context of hexagonal architecture in more detail.

Start simple

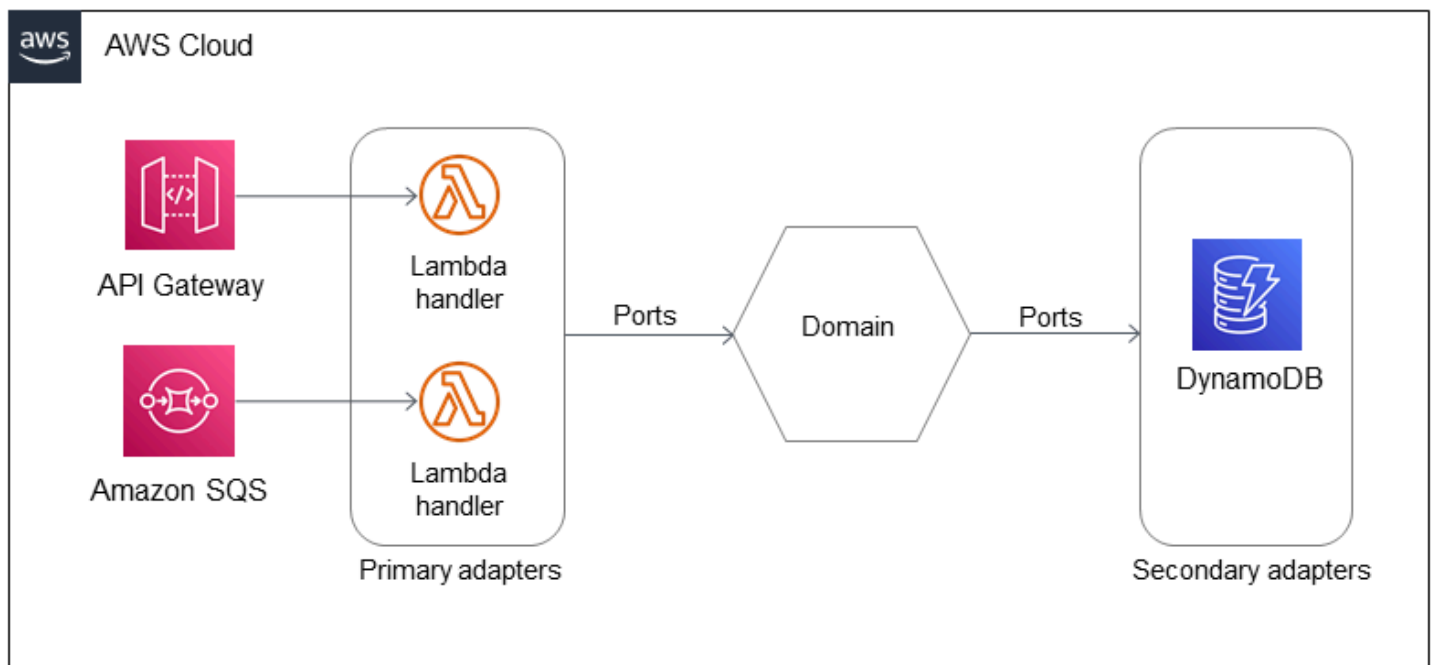
We recommend that you start simple when you architect an application by using hexagonal architecture. In this example, API Gateway is used as the client (REST API), Lambda is used as the primary adapter (compute), and DynamoDB is used as the secondary adapter (persistence). The gateway client calls the entry point, which, in this case, is a Lambda handler.



This architecture is fully serverless and gives the architect a good starting point. We recommend that you use the command pattern in the domain because it makes the code easier to maintain, and it adapts to new business and non-functional requirements. This architecture could be sufficient for building simple microservices with a few operations.

Apply the CQRS pattern

We recommend that you switch to the CQRS pattern if the number of operations on the domain is going to scale. You can apply the CQRS pattern as a fully serverless architecture in AWS by using the following example.

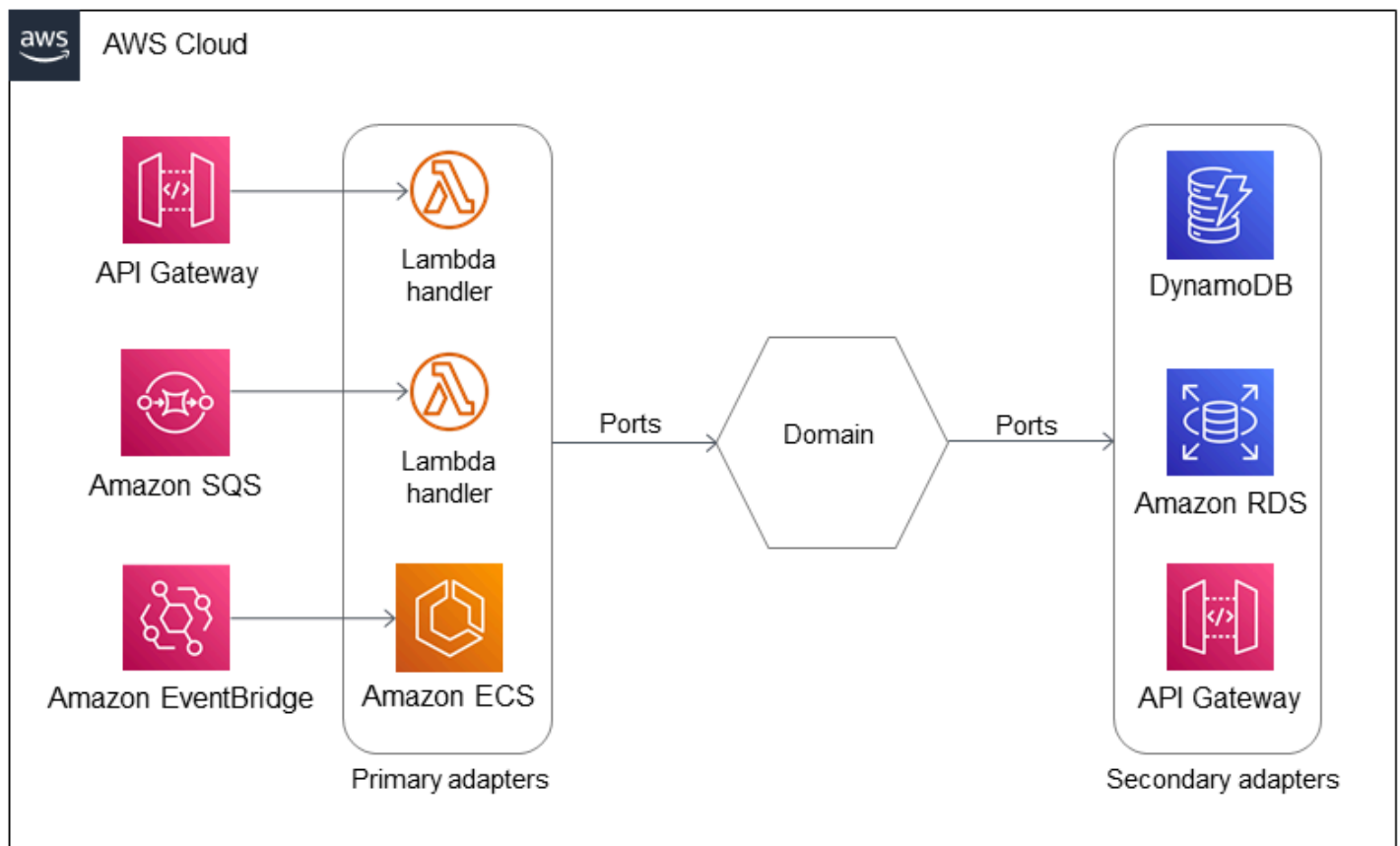


This example uses two Lambda handlers, one for queries and one for commands. Queries are run synchronously by using an API gateway as the client. Commands are run asynchronously by using Amazon SQS as the client. Amazon SQS as the client.

This architecture includes multiple clients (API Gateway and Amazon SQS) and multiple primary adapters (Lambda), which are called by their corresponding entry points (Lambda handlers). All components belong to the same bounded context, so they are within the same domain.

Evolve the architecture by adding containers, a relational database, and an external API

Containers are a good option for long-running tasks. You might also want to use a relational database if you have a predefined data schema and want to benefit from the power of the SQL language. In addition, the domain would have to communicate with external APIs. You can evolve the architecture example to support these requirements as shown in the following diagram.



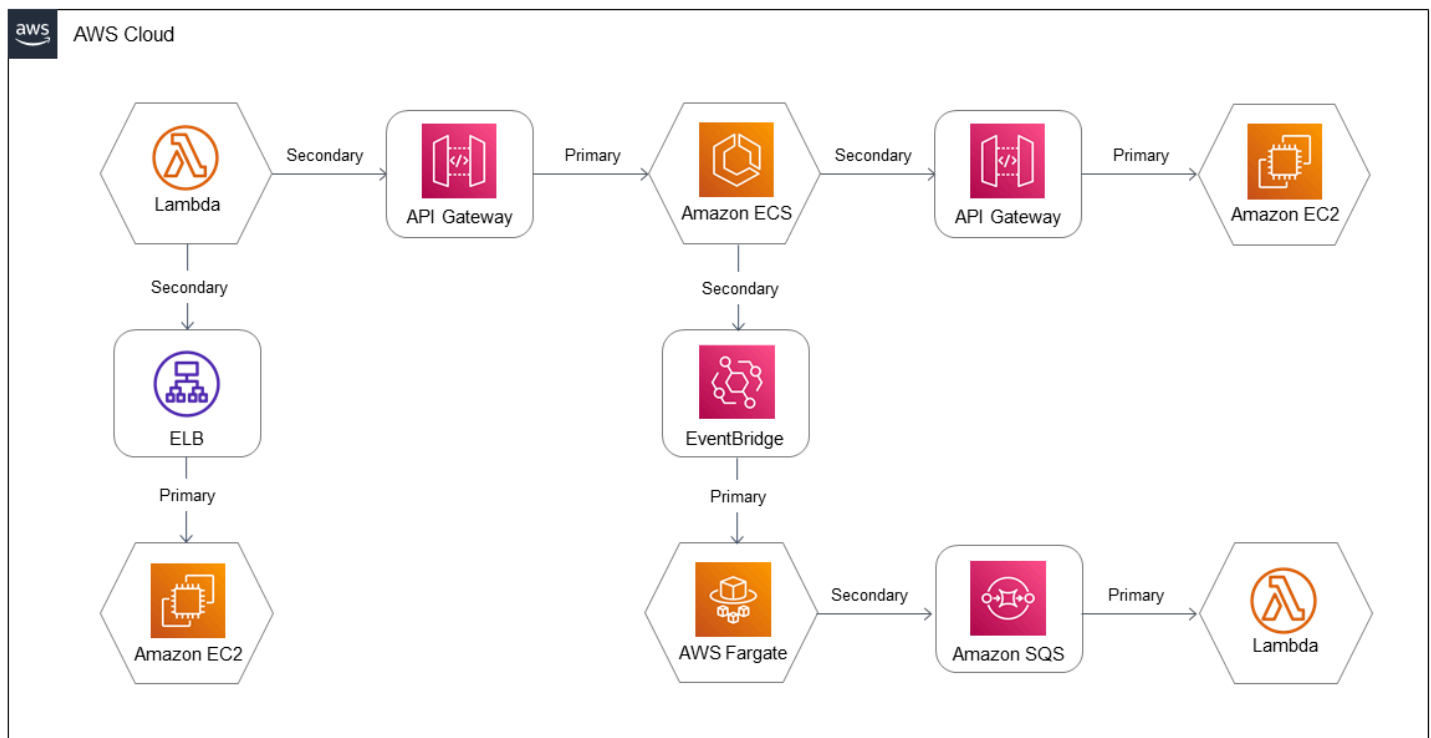
This example uses Amazon ECS as the primary adapter for launching long-running tasks in the domain. Amazon EventBridge (client) initiates an Amazon ECS task (entry point) when a specific

event happens. The architecture includes Amazon RDS as another secondary adapter for storing relational data. It also adds another API gateway as a secondary adapter for invoking an external API call. As a result, the architecture uses multiple primary and secondary adapters that rely on different underlying compute layers in one business domain.

The domain is always loosely coupled with all primary and secondary adapters through abstractions called *ports*. The domain defines what it requires from the outside world by using ports. Because it is the adapter's responsibility to implement the port, switching from one adapter to another doesn't affect the domain. For example, you can switch from Amazon DynamoDB to Amazon RDS by writing a new adapter, without affecting the domain.

Add more domains (zoom out)

Hexagonal architecture aligns well with the principles of a microservices architecture. The architecture examples shown so far contained a single domain (or bounded context). Applications typically include multiple domains, which need to communicate through primary and secondary adapters. Each domain represents a microservice and is loosely coupled with other domains.



In this architecture, each domain uses a different set of compute environment(s). (Each domain might also have multiple compute environments, as in the previous example.) Each domain defines its required interfaces to communicate with other domains through ports. Ports are implemented

by using primary and secondary adapters. This way, the domain is unaffected if there is a change in the adapter. In addition, domains are decoupled from one another.

In the architecture example shown in the previous diagram, Lambda, Amazon EC2, Amazon ECS, and AWS Fargate are used as primary adapters. API Gateway, Elastic Load Balancing, EventBridge, and Amazon SQS are used as secondary adapters.

FAQ

Why should I use a hexagonal architecture?

Hexagonal architecture shifts developers' focus to the domain logic, simplifies test automation, and improves code quality and adaptability. These improvements result in a faster time to market and easier technical and organizational scaling.

Why should I use domain-driven design?

Domain-driven design (DDD) enables you to build software components and constructs by using a common language between business stakeholders and engineers. DDD helps you manage software complexity and is an effective strategy for maintaining software products in the long term.

Can I practice test-driven development without hexagonal architecture?

Yes. Test-driven development (TDD) isn't limited to specific software design patterns. However, hexagonal architecture makes it easier to practice TDD.

Can I scale my product without hexagonal architecture and domain-driven design?

Yes. Technical and organizational product scaling can be achieved with most design patterns. However, hexagonal architecture and DDD make it easier to scale and are more effective for large projects in the long term.

Which technologies should I use to implement hexagonal architecture?

Hexagonal architecture isn't limited to a specific technology stack. We recommend that you choose technology that supports dependency inversion and unit testing.

I am developing a minimum viable product. Does it make sense to spend time thinking about software architecture?

Yes. We recommend that you use design patterns that are familiar to you for MVPs. We encourage you to try and practice hexagonal architecture until your engineers are comfortable with it. Establishing a hexagonal architecture for new projects doesn't require a significantly bigger time investment than starting without any architecture.

I am developing a minimum viable product and have no time to write tests.

If your MVP contains business logic, we strongly recommend writing automated tests for it. This will reduce the feedback loop and save time.

Which additional design patterns can I use with hexagonal architecture?

Use the [CQRS pattern](#) to support scaling of the overall system. Use the [repository pattern](#) to store and restore your domain model. Use the unit of work pattern to manage transactional process steps. Use composition over inheritance to model domain aggregates, entities, and value objects. Do not build complex object hierarchies.

Next steps

- Familiarize yourself further with domain-driven design concepts by reading the links collected in the [Resources](#) section.
- If you're implementing a new project, use the [project structure template](#) provided in this guide and implement a few features.
- If you're in the process of implementing an existing project, identify code that can be split between read-only and write-only operations. Abstract read-only code into query services, and place write-only code into command handlers.
- When your base project structure is in place, write unit tests, establish continuous integration (CI) with test automation, and follow test-driven development (TDD) practices.

Resources

References

- [Structure a Python project in hexagonal architecture using AWS Lambda](#) (AWS Prescriptive Guidance pattern)
- [Agile Teams](#) (Scaled Agile Framework website)
- [Architecture patterns with Python](#), by Harry Percival and Bob Gregory (O'Reilly Media, March 31, 2020), specifically the following chapters:
 - [Commands and Command Handler](#)
 - [Command-Query Responsibility Segregation \(CQRS\)](#)
 - [Repository Pattern](#)
- [Event storming: The smartest approach to collaboration beyond silo boundaries](#), by Alberto Brandolini (Event Storming website)
- [Demystifying Conway's Law](#), by Sam Newman (Thoughtworks website, June 30, 2014)
- [Developing evolutionary architecture with AWS Lambda](#), by James Beswick (AWS Compute Blog, July 8, 2021)
- [Domain Language: Tackling Complexity in the Heart of Software](#) (Domain Language website)
- [Facade](#), from *Dive Into Design Patterns* by Alexander Shvets (ebook, December 5, 2018)
- [GivenWhenThen](#), by Martin Fowler (August 21, 2013)
- [Implementing Domain-Driven Design](#), by Vaughn Vernon (Addison-Wesley Professional, February 2013)
- [Inverse Conway Maneuver](#) (Thoughtworks website, July 8, 2014)
- [Pattern of the Month: Red Green Refactor](#) (DZone website, June 2, 2017)
- [SOLID Design Principles Explained: Dependency Inversion Principle with Code Examples](#), by Thorben Janssen (Stackify website, May 7, 2018)
- [SOLID Principles: Explanation and examples](#), by Simon Hoiberg (ITNEXT website, Jan 1, 2019)
- [The Art of Agile Development: Test-Driven Development](#), by James Shore and Shane Warden (O'Reilly Media, March 25, 2010)
- [The SOLID Principles of Object-Oriented Programming Explained in Plain English](#), by Yigit Kemal Erinc (freeCodeCamp object-oriented programming posts, August 20, 2020)
- [What is an Event-Driven Architecture?](#) (AWS website)

AWS services

- [Amazon API Gateway](#)
- [Amazon Aurora](#)
- [Amazon DynamoDB](#)
- [Amazon Elastic Compute Cloud \(Amazon EC2\)](#)
- [Amazon Elastic Container Service \(Amazon ECS\)](#)
- [Amazon Elastic Kubernetes Service \(Amazon EKS\)](#)
- [Elastic Load Balancing](#)
- [Amazon EventBridge](#)
- [AWS Fargate](#)
- [AWS Lambda](#)
- [Amazon Relational Database Service \(Amazon RDS\)](#)
- [Amazon Simple Notification Service \(Amazon SNS\)](#)
- [Amazon Simple Queue Service \(Amazon SQS\)](#)

Other tools

- [Moto](#)
- [LocalStack](#)

Document history

The following table describes significant changes to this guide. If you want to be notified about future updates, you can subscribe to an [RSS feed](#).

Change	Description	Date
Initial publication	—	June 15, 2022